Appendix A Discretisation

The fundamental innovation of modern State Space Models (such as LSSL and S4) lies in their ability to be interpreted through three distinct, yet mathematically equivalent, lenses. As shown in Figure 1, each view offers specific computational or theoretical advantages:

1. **The Continuous View (ODE):**

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{1}$$

   The continuous view forms the theoretical foundation. It aligns with the continuous nature of real-world signals, enabling the use of HiPPO matrices to model long-range memory.

2. **The Recurrent View (RNN):**

$$x_{k+1} = \overline{A}x_k + \overline{B}u_k \tag{2}$$

   This implies a step-by-step evolution of the state. It is essential for efficient **inference** where inputs arrive sequentially.

3. **The Convolutional View (CNN):**

$$y = \overline{K} * u \tag{3}$$

   This views the system as a filter. It is critical for efficient **training**, allowing parallel computation over the entire sequence.

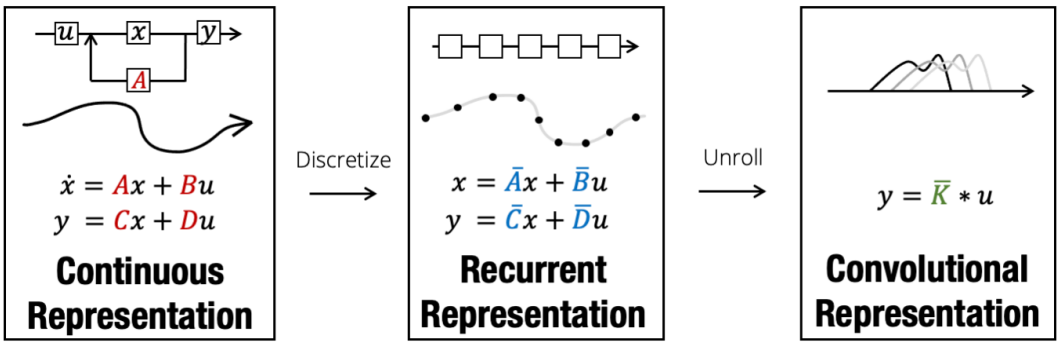

Figure 1: **The SSM Representations.** The model can be transformed from a continuous ODE (left) to a discrete recurrence (centre) and unrolled into a global convolution (right).[1]

To transfer from the theoretical **Continuous View** to **Recurrent View**, we must transform the continuous parameters (A, B) into discrete parameters $(\overline{A}, \overline{B})$ using a time step Δt. This process is termed **Discretisation**.

There is no single "correct" method to perform this transformation. In this appendix, we derive the discrete matrices by analysing the problem from three different methodological perspectives:

- **Perspective 1: Numerical Integration.** We approximate the area under the derivative curve (solving $\int \dot{x} d\tau$).
- **Perspective 2: Gradient Approximation.** We project the state forward using geometric slope approximations (Forward, Backward, and Average slopes).

- **Perspective 3: The Exact Solution (ZOH).** We derive the analytical solution of the ODE assuming a Zero-Order Hold on the input.

The following sections detail these derivations and theoretically justify why the **Bilinear (Trapezoidal)** method is the preferred choice for models like the LSSL via both empirical validation and comparisons on Taylor series expansion.

A.1 Numerical Integration

We start with the continuous-time linear time-invariant (LTI) ordinary differential equation (ODE):

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{4}$$

To determine the state evolution over the interval $[t, t + \Delta t]$, we integrate both sides of the equation:

$$\int_t^{t+\Delta t} \dot{x}(\tau)\, d\tau = \int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau))\, d\tau \tag{5}$$

Evaluating the left-hand side yields the difference between the final and initial states. For notational brevity, we define $x_k = x(t)$ and $x_{k+1} = x(t + \Delta t)$:

$$x_{k+1} - x_k = \int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau))\, d\tau \tag{6}$$

Following the integration in Eq. (6), the fundamental challenge lies in evaluating the definite integral on the right-hand side. As the trajectories of the state $x(\tau)$ and input $u(\tau)$ are unknown within the interval $\tau \in [t, t + \Delta t]$, we must employ numerical approximation methods.

We can approximate the integral $\int_t^{t+\Delta t} f(\tau)d\tau \approx \Delta t \cdot \Phi$ using different schemes for the slope Φ. This results in three distinct discretisation methods: Forward Euler, Backward Euler, and the Bilinear (Trapezoidal) transformation.

A.1.1 Forward Euler (Explicit)

The Forward Euler method approximates the area under the curve using a rectangle based on the gradient at the *start* of the interval (time t). We assume the derivatives remain constant at their initial values:

$$\int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau))\, d\tau \approx \Delta t(Ax_k + Bu_k) \tag{7}$$

$$\implies x_{k+1} - x_k = \Delta t Ax_k + \Delta t Bu_k$$

$$\implies x_{k+1} = (I + \Delta t A)x_k + \Delta t Bu_k \tag{8}$$

Note: As shown in Eq. (8), this method requires no matrix inversion but is only conditionally stable.

A.1.2 Backward Euler (Implicit)

The Backward Euler method uses the gradient at the *end* of the interval $(t + \Delta t)$. We assume values at the next time step dominate:

$$\int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau))\, d\tau \approx \Delta t(Ax_{k+1} + Bu_{k+1}) \tag{9}$$

Substituting this yields an implicit equation:

$$\begin{aligned} x_{k+1} - x_k &= \Delta t A x_{k+1} + \Delta t B u_{k+1} \\ (I - \Delta t A) x_{k+1} &= x_k + \Delta t B u_{k+1} \\ x_{k+1} &= (I - \Delta t A)^{-1} x_k + (I - \Delta t A)^{-1} \Delta t B u_{k+1} \end{aligned} \tag{10}$$

Eq. (10) shows that this method requires matrix inversion.

A.1.3 Bilinear / Trapezoidal (Implicit)

The Bilinear transform (Tustin's method) approximates the integral using the area of a trapezoid, effectively averaging the gradients at the start and end of the interval. This offers a balance between the Forward and Backward schemes and preserves the stability boundary of the continuous system.

$$\int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau))\, d\tau \approx \frac{\Delta t}{2}(Ax_k + Ax_{k+1}) + \frac{\Delta t}{2}(Bu_k + Bu_{k+1}) \tag{11}$$

Substituting this into the state equation:

$$x_{k+1} - x_k = \frac{\Delta t}{2} A(x_k + x_{k+1}) + \frac{\Delta t}{2} B(u_k + u_{k+1}) \tag{12}$$

To isolate x_{k+1}, we collect terms on the left-hand side:

$$\left(I - \frac{\Delta t}{2} A\right) x_{k+1} = \left(I + \frac{\Delta t}{2} A\right) x_k + \frac{\Delta t}{2} B(u_k + u_{k+1}) \tag{13}$$

Finally, applying the matrix inverse yields the discretised system:

$$x_{k+1} = \underbrace{\left(I - \frac{\Delta t}{2} A\right)^{-1} \left(I + \frac{\Delta t}{2} A\right)}_{\overline{A}} x_k + \underbrace{\left(I - \frac{\Delta t}{2} A\right)^{-1} \frac{\Delta t}{2} B}_{\overline{B}} (u_k + u_{k+1}) \tag{14}$$

Note on Input Handling: In many sequence modelling implementations (e.g. S4), a Zero-Order Hold (ZOH) is assumed for the input ($u(\tau) \approx u_k$). In that specific case, the input term simplifies to $(I - \frac{\Delta t}{2} A)^{-1} \Delta t B u_k$.

A.2 Geometric Derivation via Gradient Approximation

An alternative derivation can be established by examining the geometric interpretation of the derivative. Rather than integrating the area under the curve, we approximate the state trajectory using the finite difference quotient.

Recall the definition of the derivative:

$$\dot{x}(t) = \lim_{\Delta t \to 0} \frac{x(t + \Delta t) - x(t)}{\Delta t} \tag{15}$$

For a finite step size Δt, we can approximate the next state x_{k+1} by projecting from the current state x_k along a gradient vector $\mathbf{k}$:

$$x_{k+1} \approx x_k + \Delta t \cdot \mathbf{k} \tag{16}$$

The continuous ODE gives us the exact instantaneous gradient at any point: $\dot{x}(t) = Ax(t) + Bu(t)$. The difference between the discretisation methods lies entirely in *at which point in time* we choose to sample this gradient $\mathbf{k}$.

A.2.1 Forward Euler (Left-Sided Slope)

Geometric Intuition: We assume the gradient remains constant throughout the interval $[t, t + \Delta t]$, fixed at its initial value. We simply extend the tangent line at $x(t)$ forward in time.

- **Gradient Choice:** $\mathbf{k} = \dot{x}(t) = Ax_k + Bu_k$

Substituting this into Eq. (16) yields the explicit update:

$$x_{k+1} = (I + \Delta tA)x_k + \Delta tBu_k \tag{17}$$

A.2.2 Backward Euler (Right-Sided Slope)

Geometric Intuition: We assume the gradient across the interval is determined by the slope at the *destination* point $x(t+\Delta t)$. This is akin to finding a point in the future such that looking backwards with that slope leads to our current position.

- **Gradient Choice:** $\mathbf{k} = \dot{x}(t + \Delta t) = Ax_{k+1} + Bu_{k+1}$

Substituting this into the projection equation:

$$x_{k+1} = x_k + \Delta t(Ax_{k+1} + Bu_{k+1}) \tag{18}$$

Rearranging to solve for the unknown x_{k+1}:

$$\begin{aligned} x_{k+1} - \Delta tAx_{k+1} &= x_k + \Delta tBu_{k+1} \\ x_{k+1} &= (I - \Delta tA)^{-1}x_k + (I - \Delta tA)^{-1}\Delta tBu_{k+1} \end{aligned} \tag{19}$$

A.2.3 Bilinear / Trapezoidal (Average Slope)

Geometric Intuition: To achieve a more accurate approximation, we define the gradient $\mathbf{k}$ as the arithmetic mean of the initial tangent (at t) and the final tangent (at $t + \Delta t$). This compensates for the curvature of the trajectory.

- **Gradient Choice:** $\mathbf{k} = \frac{1}{2}\left(\dot{x}(t) + \dot{x}(t + \Delta t)\right)$

Substituting the ODE definitions for the slopes:

$$\mathbf{k} = \frac{1}{2}\left[(Ax_k + Bu_k) + (Ax_{k+1} + Bu_{k+1})\right] \tag{20}$$

Plugging this into the projection equation and grouping terms:

$$\begin{aligned} x_{k+1} &= x_k + \frac{\Delta t}{2}(Ax_k + Ax_{k+1}) + \frac{\Delta t}{2}(Bu_k + Bu_{k+1}) \\ \left(I - \frac{\Delta t}{2}A\right)x_{k+1} &= \left(I + \frac{\Delta t}{2}A\right)x_k + \frac{\Delta t}{2}B(u_k + u_{k+1}) \end{aligned} \tag{21}$$

Which results in the familiar Bilinear form:

$$x_{k+1} = \left(I - \frac{\Delta t}{2}A\right)^{-1}\left(I + \frac{\Delta t}{2}A\right)x_k + \dots \tag{22}$$

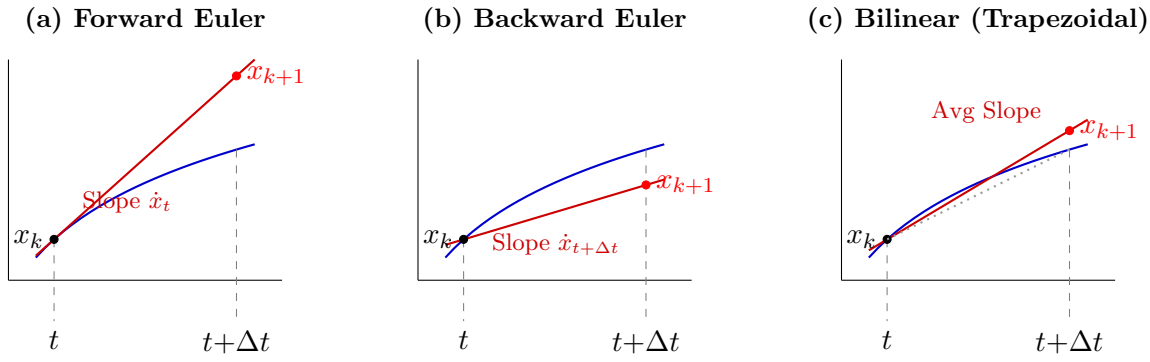

Figure 2: **Geometric Interpretation.** Comparison of gradient approximations. Forward Euler (Left) overshoots on concave curves; Backward Euler (Middle) heavily damps the step; Bilinear (Right) uses the average slope, closely tracking the curvature.

A.3 Comparative Analysis: Selecting the Optimal Discretisation

Having established three distinct discretisation methods (Forward Euler, Backward Euler, and Bilinear) via both integration and gradient approximation, a natural question arises: *Which method yields the most accurate representation of the underlying continuous dynamics?*

To answer this, we employ a two-fold evaluation strategy: first, an empirical reconstruction task to observe stability and phase properties; and second, a theoretical comparison against the analytical exact solution.

A.3.1 Empirical Validation: The Reconstruction Task

To complement the theoretical derivations, we evaluate the discretisation methods on a Memory Reconstruction task. This experiment tests the system's ability to maintain a memory of the input signal over a long duration.

We define a State Space Model with state dimensions $N \in \{64, 128, 256\}$. The task is to reconstruct a stochastic input signal $u(t)$ after a fixed time delay.

- **Input Signal:** White Noise ($\mathcal{N}(0, 1)$), chosen for its high-frequency content which stresses the stability of the discretisation.
- **Lag:** 100 time steps.
- **Sequence Length:** $L = 2000$.
- **Step Size:** $\Delta t = 0.01$.

The reconstruction performance (R^2 score) for Forward Euler, Backward Euler, and Bilinear discretisation is visualised in Figure 3. Empirical results demonstrate that whilst Forward Euler diverges due to instability and Backward Euler suffers from severe artificial damping, the Bilinear transform uniquely preserves the system's energy. This allows it to achieve perfect reconstruction ($R^2 = 1.0$) by maintaining stability without sacrificing high-frequency signal fidelity.

Although these empirical results favor the Bilinear method, visual inspection alone is insufficient. We require a mathematical ground truth to quantify the approximation error. We now turn to the derivation of the **Exact Solution** under the Zero-Order Hold (ZOH) assumption to benchmark each discretisation method.

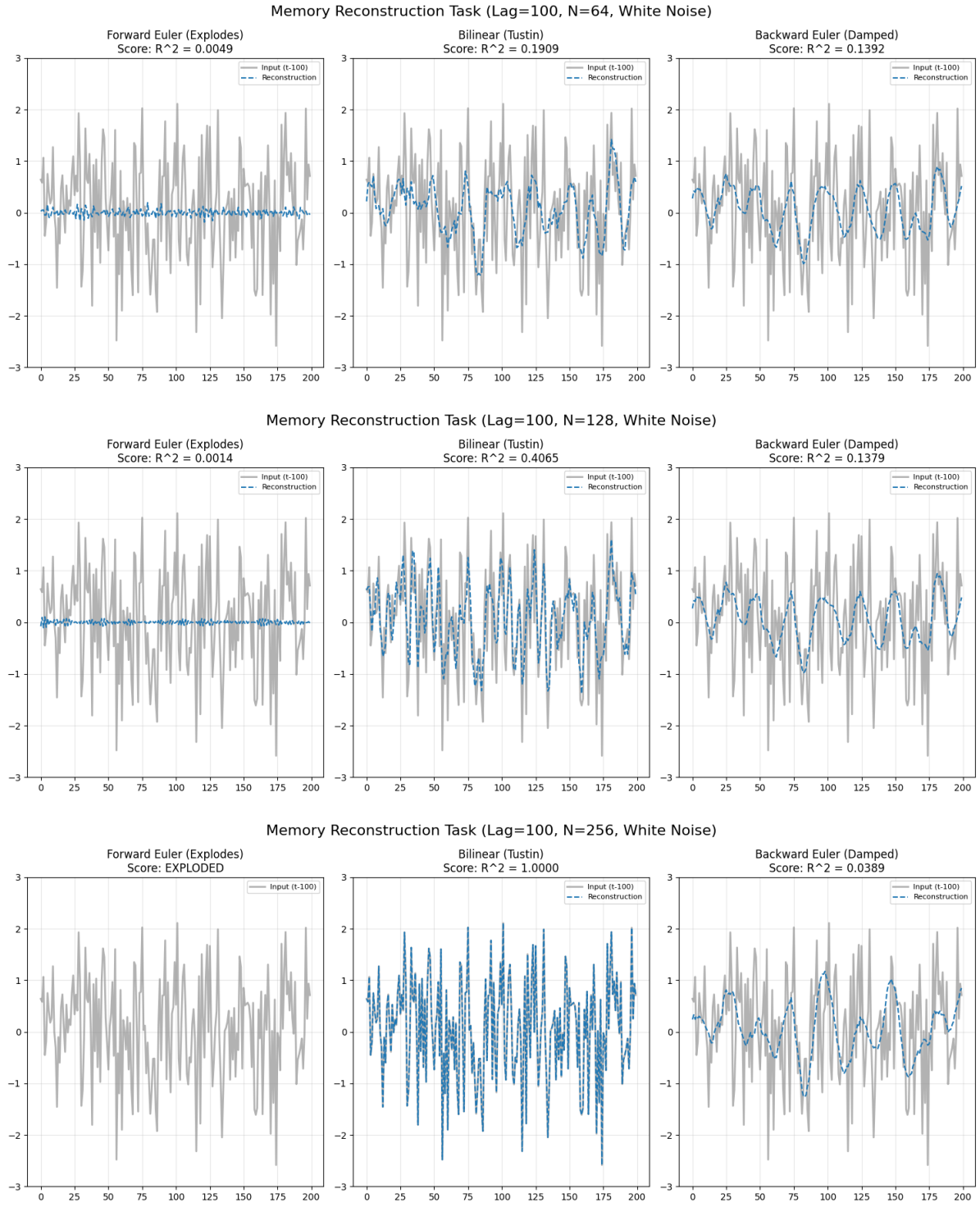

Figure 3: **Memory Reconstruction Task.** Comparison of discretisation methods across increasing state dimensions N. The blue dashed line represents the model's reconstruction of the grey input signal.

A.4 The Exact Solution (ZOH)

To mathematically benchmark the approximation methods, we derive the **Exact Solution** of the linear ODE. This serves as the third and final derivation path.

Recall the continuous ODE:

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{23}$$

Using the integrating factor e^{-At}, we can solve this differential equation analytically over the interval $[t, t + \Delta t]$. We proceed by integrating both sides:

$$\frac{d}{dt}(e^{-At}x(t)) = e^{-At}Bu(t) \tag{24}$$

$$\int_t^{t+\Delta t} \frac{d}{d\tau}(e^{-A\tau}x(\tau))\,d\tau = \int_t^{t+\Delta t} e^{-A\tau}Bu(\tau)\,d\tau$$

$$e^{-A(t+\Delta t)}x(t+\Delta t) - e^{-At}x(t) = \int_t^{t+\Delta t} e^{-A\tau}Bu(\tau)\,d\tau \tag{25}$$

Multiplying by $e^{A(t+\Delta t)}$ isolates the future state:

$$x(t+\Delta t) = \underbrace{e^{A\Delta t}}_{\overline{A}}x(t) + \int_t^{t+\Delta t} e^{A(t+\Delta t-\tau)}Bu(\tau)\,d\tau \tag{26}$$

To solve the integral involving the input $u(\tau)$, we apply the **Zero-Order Hold (ZOH) assumption**: the input is piecewise constant between sample steps ($u(\tau) = u_k$ for $\tau \in [t, t + \Delta t]$). This allows us to factor Bu_k out of the integral. Defining $\alpha = t + \Delta t - \tau$, we obtain:

$$x_{k+1} = e^{A\Delta t}x_k + \left(\int_0^{\Delta t} e^{A\alpha}\,d\alpha\right)Bu_k \tag{27}$$

Thus, the **Exact discretisation matrix** is the matrix exponential:

$$\overline{A}_{\text{exact}} = e^{A\Delta t} \tag{28}$$

A.5 Comparisons via Taylor Expansion

To theoretically justify the preference for the Bilinear method (and its adoption in architectures such as LSSL), we compare the Taylor series expansions of the discretised state transition matrices $\overline{A}$.

A.5.1 Summary of Discretisation Matrices

First, we state the state transition matrix $\overline{A}$ derived from each method. For algebraic convenience, we define $z = A\Delta t$. The transition matrices are:

- **Exact Solution (ZOH):** $\overline{A}_{\text{exact}} = e^z$.
- **Forward Euler:** $\overline{A}_{\text{FE}} = I + z$.
- **Backward Euler:** $\overline{A}_{\text{BE}} = (I - z)^{-1}$.
- **Bilinear (Tustin):** $\overline{A}_{\text{Bil}} = \left(I - \frac{1}{2}z\right)^{-1}\left(I + \frac{1}{2}z\right)$.

A.5.2 Order of Error Analysis

We now expand these matrices with respect to z to observe their approximation fidelity relative to the Exact solution.

The Benchmark: Exact Solution The Taylor series for the matrix exponential serves as our ground truth:

$$\overline{A}_{\text{exact}} = e^z = I + z + \frac{1}{2}z^2 + \frac{1}{6}z^3 + \mathcal{O}(z^4) \tag{29}$$

Euler Methods (First-Order) Comparing the Euler approximations to Eq. (29):

$$\textbf{Forward Euler:} \quad \overline{A}_{\text{FE}} = I + z \tag{30}$$

(Fails to capture $\frac{1}{2}z^2$. Error: $\mathcal{O}(\Delta t^2)$)

$$\textbf{Backward Euler:} \quad \overline{A}_{\text{BE}} = (I - z)^{-1} = I + z + \mathbf{1}z^2 + \dots \tag{31}$$

(Coefficient mismatch: 1.0 vs 0.5 at z^2 causes damping)

Bilinear / Trapezoidal (Second-Order) We use the Neumann series expansion $(I - X)^{-1} \approx I + X + X^2$, setting $X = \frac{1}{2}z$:

$$\begin{aligned}
\overline{A}_{\text{Bil}} &= \underbrace{\left(I + \frac{1}{2}z + \frac{1}{4}z^2 + \dots\right)}_{\text{Inverse Part}} \left(I + \frac{1}{2}z\right) \\
&= \left(I + \frac{1}{2}z\right) + \frac{1}{2}z\left(I + \frac{1}{2}z\right) + \frac{1}{4}z^2\left(I + \frac{1}{2}z\right) + \mathcal{O}(z^3) \\
&= I + z + \left(\frac{1}{4} + \frac{1}{4}\right)z^2 + \mathcal{O}(z^3) \\
&= I + z + \frac{1}{2}z^2 + \mathcal{O}(z^3)
\end{aligned} \tag{32}$$

Analysis: The Bilinear method matches the Exact expansion **exactly up to the second-order term** (z^2).

A.5.3 Conclusion

This derivation theoretically confirms that Bilinear discretisation is a **second-order accurate** method ($\mathcal{O}(\Delta t^3)$ local error), whereas Euler methods are only first-order accurate. This explains the superior signal reconstruction capabilities observed in the empirical tasks.

Appendix B DPLR for Matrix Inversion

The Linear State-Space Layer (LSSL) framework enables parallel training by rewriting the sequential state-space recurrence as a global convolution. Concretely, given continuous-time parameters $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ and a step size Δ, the goal is to construct the discrete convolution kernel $\bar{\mathbf{K}} = \{\mathbf{K}_m\}_{m\geq 0}$.

From the LSSL convolution kernel to a resolvent. Consider the continuous-time SSM

$$\dot{x}(t) = \mathbf{A}x(t) + \mathbf{B}u(t), \qquad y(t) = \mathbf{C}x(t),$$

whose impulse response is $k(t) = \mathbf{C}e^{t\mathbf{A}}\mathbf{B}$. After discretisation with step size Δ, we obtain a discrete-time system with matrices $(\bar{\mathbf{A}}, \bar{\mathbf{B}})$ and an induced convolution

$$y_n = \sum_{m\geq 0} \mathbf{K}_m \, u_{n-m}, \qquad \mathbf{K}_m = \mathbf{C}\,\bar{\mathbf{A}}^{\,m}\bar{\mathbf{B}}.$$

Directly forming $\bar{\mathbf{A}}^{\,m}$ is expensive. Instead, we package the entire sequence $\{\mathbf{K}_m\}$ via its z-transform (generating function)

$$\hat{\mathbf{K}}(z) \;\triangleq\; \sum_{m\geq 0} \mathbf{K}_m z^m = \mathbf{C}\Big(\sum_{m\geq 0} (\bar{\mathbf{A}}z)^m\Big)\bar{\mathbf{B}} = \mathbf{C}\,(\mathbf{I} - z\bar{\mathbf{A}})^{-1}\bar{\mathbf{B}},$$

where we used the matrix geometric series $\sum_{m\geq 0}(\bar{\mathbf{A}}z)^m = (\mathbf{I} - z\bar{\mathbf{A}})^{-1}$ (for $|z|$ sufficiently small). Therefore, computing the convolution kernel reduces to evaluating a *resolvent*-type inverse $(\mathbf{I} - z\bar{\mathbf{A}})^{-1}$ (equivalently $(z^{-1}\mathbf{I} - \bar{\mathbf{A}})^{-1}$ up to the scalar factor z^{-1}).

From $(\mathbf{I} - z\bar{\mathbf{A}})^{-1}$ to $\big(\alpha(z)\mathbf{I} - \mathbf{A}\big)^{-1}$ in S4. S4 adopts the bilinear (Tustin) discretisation, which expresses the discrete transition in terms of the original continuous-time matrix $\mathbf{A}$:

$$\bar{\mathbf{A}} = (\mathbf{I} - \tfrac{\Delta}{2}\mathbf{A})^{-1}(\mathbf{I} + \tfrac{\Delta}{2}\mathbf{A}), \qquad \bar{\mathbf{B}} = (\mathbf{I} - \tfrac{\Delta}{2}\mathbf{A})^{-1}\Delta\,\mathbf{B}.$$

Substituting this $\bar{\mathbf{A}}$ into the resolvent $(\mathbf{I} - z\bar{\mathbf{A}})^{-1}$ and simplifying yields

$$(\mathbf{I} - z\bar{\mathbf{A}})^{-1} = (\mathbf{I} - \tfrac{\Delta}{2}\mathbf{A})\Big(\alpha(z)\mathbf{I} - \mathbf{A}\Big)^{-1},$$

where the scalar mapping

$$\alpha(z) \;\triangleq\; \frac{2}{\Delta}\,\frac{1-z}{1+z}$$

depends only on z and Δ. Hence, kernel generation ultimately requires evaluating the continuous-time resolvent

$$\big(\alpha(z)\mathbf{I} - \mathbf{A}\big)^{-1}, \tag{33}$$

In S4, the resolvent is evaluated at the scalar argument $\alpha(z)$ arising from the bilinear transform. For the remainder of this appendix, we will treat this scalar argument as a free complex parameter and simplify notation by writing

$$\zeta \;\triangleq\; \alpha(z),$$

so that the core object becomes the generic resolvent $(\zeta\mathbf{I} - \mathbf{A})^{-1}$. For readability, we will subsequently drop the ζ and denote this parameter by z, i.e.,

$$(\alpha(z)\mathbf{I} - \mathbf{A})^{-1} \;\equiv\; (z\mathbf{I} - \mathbf{A})^{-1},$$

with the understanding that this z stands for the scalar input to the resolvent (not necessarily the original z-transform variable).

To illustrate how the DPLR/NPLR structure reduces this cost, we now unpack the algebra step-by-step (and, for intuition, we will later show a small $N = 3$ instance).

B.1 General Matrix Inversion and $\mathcal{O}(N^3)$

Consider a generic dense symmetric state matrix $\mathbf{M} \in \mathbb{R}^{3\times3}$, initialised as:

$$\mathbf{M} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \tag{34}$$

Computing $\mathbf{M}^{-1}$ typically requires Gaussian elimination. This process transforms $\mathbf{M}$ into the Identity matrix $\mathbf{I}$ through a sequence of row operations; applying these same operations to $\mathbf{I}$ yields $\mathbf{M}^{-1}$.

We proceed by constructing the **augmented matrix** $[\mathbf{M}|\mathbf{I}]$:

$$[\mathbf{M}|\mathbf{I}] = \left[\begin{array}{ccc|ccc} 2 & 1 & 1 & 1 & 0 & 0 \\ 1 & 2 & 1 & 0 & 1 & 0 \\ 1 & 1 & 2 & 0 & 0 & 1 \end{array}\right] \tag{35}$$

B.1.1 Forward Elimination (Gaussian Steps)

Our goal is to introduce zeros below the main diagonal to create an upper triangular matrix. First, we eliminate the entries below the first pivot ($M_{1,1} = 2$). We subtract half of Row 1 (R_1) from Row 2 (R_2) and Row 3 (R_3):

$$R_2 \leftarrow R_2 - 0.5R_1$$
$$R_3 \leftarrow R_3 - 0.5R_1$$

This updates the entire row, including the augmented side:

$$\left[\begin{array}{ccc|ccc} 2 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1.5 & 0.5 & -0.5 & 1 & 0 \\ 0 & 0.5 & 1.5 & -0.5 & 0 & 1 \end{array}\right] \tag{36}$$

Next, we eliminate the entry below the second pivot ($M_{2,2} = 1.5$). We subtract $\frac{1}{3}$ of Row 2 from Row 3 ($0.5/1.5 = 1/3$):

$$R_3 \leftarrow R_3 - \frac{1}{3}R_2$$

This yields the upper triangular form:

$$\left[\begin{array}{ccc|ccc} 2 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1.5 & 0.5 & -0.5 & 1 & 0 \\ 0 & 0 & 1.33 & -0.33 & -0.33 & 1 \end{array}\right] \tag{37}$$

B.1.2 Back Substitution (Jordan Steps)

Now that the system is triangular, we normalise the pivots to 1 and eliminate the entries above the diagonal.

Starting from the bottom row (R_3), we divide by the pivot (1.33 or 4/3) to isolate the identity component:

$$R_3 \leftarrow R_3 \times \frac{3}{4} \implies [0, 0, 1 \mid -0.25, -0.25, 0.75] \tag{38}$$

We then substitute R_3 upwards to eliminate the entries above it in R_2 and R_1, and finally normalise R_2 and R_1. After performing these operations for all rows, the left side becomes the Identity matrix, and the right side becomes $\mathbf{M}^{-1}$:

$$\mathbf{M}^{-1} = \begin{bmatrix} 0.75 & -0.25 & -0.25 \\ -0.25 & 0.75 & -0.25 \\ -0.25 & -0.25 & 0.75 \end{bmatrix} \tag{39}$$

B.1.3 Complexity Analysis

Notice that every row operation ($R_i \leftarrow R_i - cR_j$) requires updating **every column** in the row.

- For a matrix of size N, there are N rows.
- Each row elimination requires $\approx N$ multiplications and subtractions.
- This process is repeated N times for the columns.

The total operation count scales as $\approx \frac{2}{3}N^3$. For standard state dimensions (e.g., $N = 64$), this cost is 2.6×10^5 operations. Crucially, the LSSL framework requires this evaluation at every frequency step L. The resulting total complexity of $\mathcal{O}(LN^3)$ renders this naive approach computationally infeasible for long sequences (e.g., $L \geq 16,000$).

B.2 The Ideal Case: Matrix Inversion for Diagonal Matrices and $\mathcal{O}(N)$

Consider the scenario where the state matrix is strictly diagonal. Let $\mathbf{\Lambda}$ be a diagonal matrix:

$$\mathbf{\Lambda} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} \tag{40}$$

The inversion of a diagonal matrix is trivial; one simply computes the reciprocal of the diagonal elements:

$$\mathbf{\Lambda}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1/2 & 0 \\ 0 & 0 & 1/3 \end{bmatrix} \tag{41}$$

Complexity Analysis This operation requires exactly N divisions, resulting in $\mathcal{O}(N)$ complexity. While computationally ideal, diagonal matrices imply that the state variables are independent, severely limiting the model's capacity to capture complex interactions or memory.

B.3 Bridging the Gap: Structured Inversion via DPLR

To circumvent the cubic bottleneck while retaining expressivity, we constrain the state matrix to the **Diagonal Plus Low-Rank (DPLR)** class. This structure admits a closed-form inverse via the Woodbury Matrix Identity, reducing the problem to simple vector arithmetic.

We observe that the matrix $\mathbf{M}$ from Section B.1 decomposes into a diagonal term $\mathbf{\Lambda}$ and a symmetric rank-1 correction derived from the vector $\mathbf{p}$:

$$\mathbf{M} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{\Lambda}} + \underbrace{\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}}_{\mathbf{p}} \underbrace{\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}}_{\mathbf{p}^T} \tag{42}$$

where $\mathbf{\Lambda} = \mathbf{I}$ and $\mathbf{p} = [1, 1, 1]^T$. The Woodbury identity for a symmetric rank-1 correction is given by:

$$(\mathbf{\Lambda} + \mathbf{p}\mathbf{p}^T)^{-1} = \mathbf{\Lambda}^{-1} - \frac{\mathbf{\Lambda}^{-1}\mathbf{p}\mathbf{p}^T\mathbf{\Lambda}^{-1}}{1 + \mathbf{p}^T\mathbf{\Lambda}^{-1}\mathbf{p}} \tag{43}$$

Applying this directly to our matrix $\mathbf{M}$, the computation proceeds as follows:

1. **Diagonal Inversion:** Computing $\mathbf{\Lambda}^{-1}$ requires only element-wise division. Here, $\mathbf{\Lambda}^{-1} = \mathbf{I}$ ($\mathcal{O}(N)$).

2. **Matrix-Vector Product:** We compute the intermediate vector $\mathbf{w} = \mathbf{\Lambda}^{-1}\mathbf{p}$. Since $\mathbf{\Lambda}^{-1}$ is identity, $\mathbf{w} = [1, 1, 1]^T$ ($\mathcal{O}(N)$).

3. **Scalar Reduction:** The denominator term is computed as a dot product: $k = 1 + \mathbf{p}^T\mathbf{w} = 1 + 3 = 4$ ($\mathcal{O}(N)$).

4. **Low-Rank Update:** The correction term is derived via the outer product of $\mathbf{w}$:

$$\frac{\mathbf{w}\mathbf{w}^T}{k} = \frac{1}{4}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix} = \begin{bmatrix}0.25 & 0.25 & 0.25\\ 0.25 & 0.25 & 0.25\\ 0.25 & 0.25 & 0.25\end{bmatrix} \tag{44}$$

5. **Final Assembly**:

$$\mathbf{M}^{-1} = \mathbf{\Lambda}^{-1} - \text{Correction} = \begin{bmatrix}0.75 & -0.25 & -0.25\\ -0.25 & 0.75 & -0.25\\ -0.25 & -0.25 & 0.75\end{bmatrix} \tag{45}$$

This derivation recovers the exact result of the Gaussian elimination but reduces the arithmetic complexity from $\mathcal{O}(N^3)$ to $\mathcal{O}(N)$.

B.4 Extend DPLR to HiPPO Matrix

The previous section demonstrated that if a matrix has the structure $\mathbf{M} = \mathbf{\Lambda} + \mathbf{p}\mathbf{p}^T$ (Diagonal Plus Low-Rank), it admits computationally efficient inversion. However, the state matrix $\mathbf{A}$ in S4 is derived from the **HiPPO** (High-order Polynomial Projection Operators) framework to optimally compress history. We now follow the derivation from S4 and show how the HiPPO-LegS matrix can be converted into a **Normal Plus Low-Rank** form, and hence into a DPLR matrix after a unitary change of basis.

B.4.1 The Definition of the HiPPO-LegS Matrix

We focus on the HiPPO-LegS (Legendre Scaled) matrix, the default initialisation for S4. It is defined element-wise as:

$$A_{nk} = \begin{cases} -(2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ -(n+1) & \text{if } n = k \\ 0 & \text{if } n < k \end{cases} \tag{46}$$

A naive inspection reveals that $\mathbf{A}$ is a **dense, lower-triangular matrix**. Standard inversion would incur the prohibitively expensive $\mathcal{O}(N^3)$ cost discussed in Section B.1.

B.4.2 Revealing the Hidden Structure (NPLR)

Although the matrix appears strictly triangular, S4 observes that adding a specific rank-1 term reveals a hidden *normal* structure. Let $\mathbf{p} \in \mathbb{R}^N$ with entries

$$p_n = \sqrt{2n+1}. \tag{47}$$

Define the rank-1 update

$$\widehat{\mathbf{A}} \triangleq \mathbf{A} + \frac{1}{2}\mathbf{p}\mathbf{p}^T. \tag{48}$$

We now define a strictly skew-symmetric matrix $\mathbf{S}$ element-wise by

$$S_{nk} = \begin{cases} \frac{1}{2}p_n p_k, & n < k, \\ 0, & n = k, \\ -\frac{1}{2}p_n p_k, & n > k. \end{cases} \tag{49}$$

By construction, $\mathbf{S}^T = -\mathbf{S}$. A direct entry-wise check shows that $\widehat{\mathbf{A}}$ becomes "constant plus skew":

$$\widehat{\mathbf{A}} = -\frac{1}{2}\mathbf{I} - \mathbf{S}. \tag{50}$$

Equivalently, substituting (48) into (50) yields the NPLR decomposition of the original HiPPO matrix:

$$\mathbf{A} = -\mathbf{S} - \frac{1}{2}\mathbf{p}\mathbf{p}^T - \frac{1}{2}\mathbf{I}. \tag{51}$$

$$\underbrace{\begin{bmatrix} \bullet & 0 & \cdots \\ -p_1p_0 & \bullet & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}}_{\mathbf{A}\text{ (Lower Triangular)}} = -\underbrace{\begin{bmatrix} 0 & -\frac{1}{2}p_0p_1 & \cdots \\ +\frac{1}{2}p_1p_0 & 0 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}}_{\mathbf{S}\text{ (Skew-Symmetric)}} - \frac{1}{2}\underbrace{\begin{bmatrix} p_0^2 & p_0p_1 & \cdots \\ p_1p_0 & p_1^2 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}}_{\mathbf{p}\mathbf{p}^T\text{ (Dense Symmetric)}} - \frac{1}{2}\underbrace{\begin{bmatrix} 1 & 0 & \cdots \\ 0 & 1 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}}_{\mathbf{I}} \tag{52}$$

We verify (50) (and hence (51)) by checking the three cases:

- **Upper Triangle ($n < k$):** $A_{nk} = 0$, while $\left(\frac{1}{2}\mathbf{p}\mathbf{p}^T\right)_{nk} = +\frac{1}{2}p_np_k$, so $\widehat{A}_{nk} = +\frac{1}{2}p_np_k$. From (49), $S_{nk} = +\frac{1}{2}p_np_k$, so $(-\frac{1}{2}\mathbf{I} - \mathbf{S})_{nk} = -S_{nk} = +\frac{1}{2}p_np_k$ matches.
- **Lower Triangle ($n > k$):** $A_{nk} = -p_np_k$, while $\left(\frac{1}{2}\mathbf{p}\mathbf{p}^T\right)_{nk} = +\frac{1}{2}p_np_k$, so $\widehat{A}_{nk} = -\frac{1}{2}p_np_k$. From (49), $S_{nk} = -\frac{1}{2}p_np_k$, so $(-\frac{1}{2}\mathbf{I} - \mathbf{S})_{nk} = -S_{nk} = -\frac{1}{2}p_np_k$ matches.
- **Diagonal ($n = k$):** $\widehat{A}_{nn} = A_{nn} + \frac{1}{2}p_n^2 = -(n+1) + \frac{1}{2}(2n+1) = -\frac{1}{2}$, while $(-\frac{1}{2}\mathbf{I} - \mathbf{S})_{nn} = -\frac{1}{2}$ since $S_{nn} = 0$.

B.4.3 Unitary Diagonalisation

Normality and unitary diagonalisation. Since $\mathbf{S}^T = -\mathbf{S}$, we have $\mathbf{S}\mathbf{S}^T = \mathbf{S}^T\mathbf{S}$ and thus $\mathbf{S}$ is normal.[1] By the spectral theorem for normal matrices, there exists a unitary matrix $\mathbf{V} \in \mathbb{C}^{N\times N}$ ($\mathbf{V}^*\mathbf{V} = \mathbf{I}$) that diagonalises $\mathbf{S}$:

$$\mathbf{S} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^*. \tag{53}$$

Why a similarity transform? Our ultimate goal is to evaluate resolvents of the form $(z\mathbf{I} - \mathbf{A})^{-1}$. Diagonalising the normal component $\mathbf{S}$ suggests working in the $\mathbf{V}$-basis, where the normal part becomes diagonal, reducing the core computation to a diagonal-plus-low-rank inverse amenable to Woodbury. Because $\mathbf{V}$ is unitary, this change of basis is numerically stable (it preserves norms) and does not alter the underlying operator.

Definition (conjugation). We define the conjugated matrix

$$\tilde{\mathbf{A}} \triangleq \mathbf{V}^*\mathbf{A}\mathbf{V}. \tag{54}$$

This definition is natural because it implies the exact identity

$$\mathbf{A} = \mathbf{V}\tilde{\mathbf{A}}\mathbf{V}^*, \tag{55}$$

obtained by inserting $\mathbf{I} = \mathbf{V}\mathbf{V}^*$ on both sides: $\mathbf{A} = (\mathbf{V}\mathbf{V}^*)\mathbf{A}(\mathbf{V}\mathbf{V}^*) = \mathbf{V}(\mathbf{V}^*\mathbf{A}\mathbf{V})\mathbf{V}^*$.

[1] Equivalently, one may work over $\mathbb{C}$ and use the skew-Hermitian relation $\mathbf{S}^* = -\mathbf{S}$, which also implies normality.

Inverse under conjugation. For any invertible matrix $\mathbf{M}$ and any unitary $\mathbf{V}$,

$$(\mathbf{V}\mathbf{M}\mathbf{V}^*)^{-1} = \mathbf{V}\mathbf{M}^{-1}\mathbf{V}^*, \tag{56}$$

which follows by direct multiplication: $(\mathbf{V}\mathbf{M}\mathbf{V}^*)(\mathbf{V}\mathbf{M}^{-1}\mathbf{V}^*) = \mathbf{V}\mathbf{I}\mathbf{V}^* = \mathbf{I}$.

Corollary (resolvent under conjugation). Recall from (55) and that $\mathbf{V}$ is unitary. Then

$$\begin{aligned} z\mathbf{I} - \mathbf{A} &= z\mathbf{I} - \mathbf{V}\tilde{\mathbf{A}}\mathbf{V}^* \\ &= z(\mathbf{V}\mathbf{V}^*) - \mathbf{V}\tilde{\mathbf{A}}\mathbf{V}^* \qquad (\text{since } \mathbf{I} = \mathbf{V}\mathbf{V}^*) \\ &= \mathbf{V}(z\mathbf{I})\mathbf{V}^* - \mathbf{V}\tilde{\mathbf{A}}\mathbf{V}^* \\ &= \mathbf{V}\big(z\mathbf{I} - \tilde{\mathbf{A}}\big)\mathbf{V}^*. \end{aligned} \tag{57}$$

Finally, applying the inverse-under-conjugation identity (56) with $\mathbf{M} = z\mathbf{I} - \tilde{\mathbf{A}}$ yields

$$(z\mathbf{I} - \mathbf{A})^{-1} = \big(\mathbf{V}(z\mathbf{I} - \tilde{\mathbf{A}})\mathbf{V}^*\big)^{-1} = \mathbf{V}(z\mathbf{I} - \tilde{\mathbf{A}})^{-1}\mathbf{V}^*. \tag{58}$$

so it suffices to compute the resolvent of the conjugated matrix $\tilde{\mathbf{A}}$.

Obtaining the DPLR form. Substituting (51) gives

$$\begin{aligned} \tilde{\mathbf{A}} &= \mathbf{V}^* \left(-\mathbf{S} - \frac{1}{2}\mathbf{p}\mathbf{p}^T - \frac{1}{2}\mathbf{I}\right)\mathbf{V} \\ &= \underbrace{-\mathbf{V}^*\mathbf{S}\mathbf{V}}_{\text{Term 1}} - \underbrace{\frac{1}{2}\mathbf{V}^*(\mathbf{p}\mathbf{p}^T)\mathbf{V}}_{\text{Term 2}} - \underbrace{\frac{1}{2}\mathbf{I}}_{\text{Term 3}}. \end{aligned} \tag{59}$$

Analyzing the terms:

- **Term 1:** From (53), $\mathbf{V}^*\mathbf{S}\mathbf{V} = \mathbf{\Lambda}$ (diagonal). Hence

  $$-\mathbf{V}^*\mathbf{S}\mathbf{V} = -\mathbf{\Lambda}.$$

- **Term 2:** Define $\tilde{\mathbf{p}} = \mathbf{V}^*\mathbf{p}$. Since $\mathbf{p} \in \mathbb{R}^N$, we have $\mathbf{p}^T = \mathbf{p}^*$ and therefore $\mathbf{p}^T\mathbf{V} = (\mathbf{V}^*\mathbf{p})^* = \tilde{\mathbf{p}}^*$. Unitary conjugation preserves rank, and

  $$\frac{1}{2}\mathbf{V}^*(\mathbf{p}\mathbf{p}^T)\mathbf{V} = \frac{1}{2}(\mathbf{V}^*\mathbf{p})(\mathbf{p}^T\mathbf{V}) = \frac{1}{2}\tilde{\mathbf{p}}\tilde{\mathbf{p}}^*, \tag{60}$$

  a rank-1 correction.

- **Term 3:** The shift $-\frac{1}{2}\mathbf{I}$ is diagonal and does not change the diagonal-plus-low-rank structure.

Thus, the effective matrix is **DPLR**:

$$\tilde{\mathbf{A}} = -\mathbf{\Lambda} - \frac{1}{2}\tilde{\mathbf{p}}\tilde{\mathbf{p}}^* - \frac{1}{2}\mathbf{I}. \tag{61}$$

This shows that the dense HiPPO-LegS matrix is unitarily equivalent to a diagonal matrix plus a rank-1 correction (up to a diagonal shift). Consequently, computing $(z\mathbf{I} - \mathbf{A})^{-1}$ reduces to computing $(z\mathbf{I} - \tilde{\mathbf{A}})^{-1}$ via (58), where the core inverse is diagonal-plus-rank-1 and can be evaluated in $\mathcal{O}(N)$ using Woodbury (Section B.3).

B.5 Closing the Loop: Evaluating the Resolvent $(z\mathbf{I} - \mathbf{A})^{-1}$

Recall from Eq. (33) that our ultimate objective is not merely to invert $\mathbf{A}$, but to evaluate the resolvent $(z\mathbf{I} - \mathbf{A})^{-1}$ to compute the convolution kernel. We can now demonstrate that the DPLR/NPLR structure derived above renders this operation computationally tractable.

From Section B.4, we have the unitary change of basis $\tilde{\mathbf{A}} = \mathbf{V}^*\mathbf{A}\mathbf{V}$ with

$$\tilde{\mathbf{A}} = -\mathbf{\Lambda} - \frac{1}{2}\tilde{\mathbf{p}}\tilde{\mathbf{p}}^* - \frac{1}{2}\mathbf{I}, \qquad \tilde{\mathbf{p}} = \mathbf{V}^*\mathbf{p}, \tag{62}$$

and $\mathbf{A} = \mathbf{V}\tilde{\mathbf{A}}\mathbf{V}^*$.

B.5.1 Reducing the Resolvent to "Diagonal + Rank-1"

Using the unitary invariance identity $(\mathbf{V}\mathbf{M}\mathbf{V}^*)^{-1} = \mathbf{V}\mathbf{M}^{-1}\mathbf{V}^*$, we obtain

$$\begin{aligned}(z\mathbf{I} - \mathbf{A})^{-1} &= \left(z\mathbf{I} - \mathbf{V}\tilde{\mathbf{A}}\mathbf{V}^*\right)^{-1} \\ &= \mathbf{V}\left(z\mathbf{I} - \tilde{\mathbf{A}}\right)^{-1}\mathbf{V}^*.\end{aligned} \tag{63}$$

Substituting (62) gives the core form

$$\begin{aligned}\left(z\mathbf{I} - \tilde{\mathbf{A}}\right)^{-1} &= \left(z\mathbf{I} + \mathbf{\Lambda} + \frac{1}{2}\tilde{\mathbf{p}}\tilde{\mathbf{p}}^* + \frac{1}{2}\mathbf{I}\right)^{-1} \\ &= \underbrace{\left(\left(\left(z + \frac{1}{2}\right)\mathbf{I} + \mathbf{\Lambda}\right) + \frac{1}{2}\tilde{\mathbf{p}}\tilde{\mathbf{p}}^*\right)^{-1}}_{\text{Diagonal + rank-1}}.\end{aligned} \tag{64}$$

Therefore,

$$(z\mathbf{I} - \mathbf{A})^{-1} = \mathbf{V}\left(\left(\left(z + \frac{1}{2}\right)\mathbf{I} + \mathbf{\Lambda}\right) + \frac{1}{2}\tilde{\mathbf{p}}\tilde{\mathbf{p}}^*\right)^{-1}\mathbf{V}^*. \tag{65}$$

The term inside the brackets is strictly the inverse of a diagonal matrix plus a rank-1 correction, matching the form required for the Woodbury identity and enabling $\mathcal{O}(N)$ evaluation per z.

B.5.2 The Computational Advantage

Let

$$\mathbf{D} \triangleq \left(z + \frac{1}{2}\right)\mathbf{I} + \mathbf{\Lambda}. \tag{66}$$

Since $\mathbf{\Lambda}$ is diagonal and z is a scalar, $\mathbf{D}$ is strictly diagonal. The problem has now been reduced to

$$\left(\mathbf{D} + \frac{1}{2}\tilde{\mathbf{p}}\tilde{\mathbf{p}}^*\right)^{-1}, \tag{67}$$

which is exactly the "diagonal + rank-1" case solved in Section B.3.

By applying the Woodbury Matrix Identity, we avoid explicit matrix inversion:

1. **Diagonal Inversion:** We compute $\mathbf{D}^{-1} = \text{diag}\left(1/\left(z + \frac{1}{2} + \lambda_i\right)\right)$. This is $\mathcal{O}(N)$.
2. **Woodbury Correction:** We apply Eq. (43) with $\mathbf{D}^{-1}$ and $\tilde{\mathbf{p}}$, which involves only vector dot products and scaling and is $\mathcal{O}(N)$.

Thus, evaluating the core resolvent term (64) is reduced from a cubic $\mathcal{O}(N^3)$ operation to a linear $\mathcal{O}(N)$ operation (per z), and the full resolvent is recovered via the surrounding unitary transforms in (65).

B.5.3 Impact on Global Complexity

The $\mathcal{O}(N)$ per-parameter resolvent evaluation derived above is a prerequisite for the overall efficiency of S4: it reduces the core linear-algebraic bottleneck from cubic to linear time for each resolvent query. In practice, constructing the full convolution kernel requires evaluating this resolvent at many complex parameters. Appendix C will introduce how S4 organises these evaluations and accelerates them using frequency-domain structure, ultimately yielding the $\mathcal{O}(N \log N)$ kernel computation.

Appendix C The Evolution of the State Matrix A

While Appendix A established the necessity of time-variant parameters for selective memory, it is crucial to understand the structural evolution of the state matrix A. The matrix A governs the internal memory dynamics—determining how past information is retained, mixed, or forgotten.

The development of modern SSMs can be traced through the structural changes applied to A to balance expressiveness (memory capacity) with computational efficiency. This evolution follows the lineage of HiPPO, LSSL, S4, Mamba-01, and Mamba-02.

C.1 HiPPO: Optimal History Compression

The High-Order Polynomial Projection Operators (HiPPO) framework addresses the fundamental question: *How can a system compress an infinite continuous history $x(t)$ into a finite hidden state $h(t)$?*

HiPPO achieves this by projecting the input signal onto a basis of orthogonal polynomials (specifically, scaled Legendre polynomials). The resulting state matrix A is derived mathematically to minimise the reconstruction error of the history. In the standard Legendre case (LegS), the matrix $A \in \mathbb{R}^{N\times N}$ is defined element-wise as:

$$A_{nk} = -\begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases} \tag{68}$$

This results in a dense, lower-triangular matrix. While theoretically optimal for memory retention, the dense nature of A makes computation expensive ($O(N^2)$ for matrix-vector multiplication), which hinders scaling to deep networks.

C.2 LSSL: Discretisation and the Bilinear Transform

The Linear State-Space Layer (LSSL) bridges the continuous HiPPO theory with discrete sequence modelling. It utilises the HiPPO matrix as an initialisation inductive bias and applies Bilinear (Tustin) discretisation to convert the continuous system into a discrete recurrence:

$$\overline{A} = (I - \frac{\Delta}{2}A)^{-1}(I + \frac{\Delta}{2}A) \tag{69}$$

At this stage, the system is still Linear Time Invariant (LTI). The primary contribution of LSSL is establishing the dual view of LTI SSMs: they can be trained in parallel like Convolutional Neural Networks (CNNs) and inferenced sequentially like Recurrent Neural Networks (RNNs).

C.3 S4: Diagonal Plus Low-Rank (DPLR)

The Structured State Space sequence model (S4) solves the computational bottleneck of LSSL. Inverting the matrix $(I-\frac{\Delta}{2}A)$ for a dense HiPPO matrix is computationally prohibitive ($O(N^3)$).

S4 introduces the DPLR parametrisation. It proves that the HiPPO matrix can be decomposed into a Normal (diagonal) component plus a Low-Rank correction:

$$A = \Lambda + PQ^\top \tag{70}$$

where Λ is diagonal and $P, Q \in \mathbb{R}^{N\times r}$ (typically $r = 1$).

By exploiting the Woodbury Matrix Identity, the inversion of a DPLR matrix can be reduced to inverting a diagonal matrix and a small $r \times r$ matrix. This reduces the computational complexity from $O(N^3)$ to roughly $O(N)$, allowing SSMs to scale efficiently to long sequences while retaining the memory properties of HiPPO.

C.4 Mamba-01: Time-Variant Diagonal A

Mamba-01 marks the transition from LTI to Selective SSMs. As discussed in Appendix A, pure LTI systems cannot perform content-aware filtering. Mamba introduces an input-dependent step size Δ_t.

To maintain efficiency with a time-varying system, Mamba-01 simplifies the structure of A further to be purely diagonal. The complex DPLR structure is often replaced with diagonal initialisations that approximate HiPPO dynamics (e.g., $A_n = -1/2 + ni$).

Under Zero-Order Hold (ZOH) discretisation, the effective discrete transition becomes time-dependent:

$$\overline{A}_t = \exp(\Delta_t A) \tag{71}$$

Since Δ_t depends on the input x_t, the system can selectively "reset" state channels (large Δ_t) or "freeze" memories (small Δ_t) per token.

C.5 Mamba-02: State Space Duality (SSD) and Scalar Identity

Mamba-02 addresses the hardware efficiency of SSMs on GPUs (which are optimised for large matrix multiplications) and the conceptual link to the Transformer architecture. It introduces **State Space Duality (SSD), reframing the SSM recurrence as a matrix operator $Y = MX$.

For the matrix operator M to be computed efficiently and resemble the Attention mechanism, Mamba-02 proposes a Scalar Identity structure for A:

$$A_t = a_t I \tag{72}$$

where a_t is a scalar. When the recurrence is unrolled, the interaction between time steps j and i (where $j \geq i$) simplifies to:

$$M_{ji} = (C_j^\top B_i) \cdot \prod_{k=i+1}^{j} a_k \tag{73}$$

This reveals a direct structural mapping to Causal Linear Attention:

- $C_j \leftrightarrow Q$ (Query)
- $B_i \leftrightarrow K$ (Key)
- $x_i \leftrightarrow V$ (Value)
- The product of scalars $\prod a_k$ acts as a data-dependent decay mask (similar to positional encoding or attention masking).

This formulation allows Mamba-02 to utilise the highly optimised kernel implementations used for Attention, bridging the gap between SSMs and Transformers.

Appendix D From Linear Time Invariant (LTI) to Selective State Space Models

This part of the appendix details the theoretical transition from Linear Time Invariant (LTI) systems to Time-Variant State Space Models (specifically the Mamba architecture). We utilise a simplified illustrative task—memorising London's weather pattern—to demonstrate the limitations of LTI memory and the necessity for input-dependent parametrisation.

D.1 The Limitation of LTI Systems: The London Weather Analogy

To motivate the transition from *linear time-invariant* (LTI) state space models (SSMs) to *time-varying* SSMs (as used by Mamba-style selective mechanisms), we consider a simple sequence modelling task: retaining a memory of London's weather over a ten-day period. Let $t \in \{0, 1, \ldots, 9\}$ denote the time step. We define:

- x_t: the input at day t (the observed weather/event);
- h_t: the hidden state (the model's memory) accumulated up to day t;
- y_t: the output at day t (e.g. a decision to carry an umbrella, where 1 = yes and 0 = no).

Intuition: memory is updated by the past and the present. In our hypothetical sequence, most days are rainy with occasional sunshine. At $t = 5$, however, a highly salient anomaly occurs: an "Alien Invasion". Intuitively, the memory state h_5 should reflect both the accumulated context h_4 and the novel input x_5. In other words, the update should be sensitive to the *content* of the new observation: mundane, repetitive inputs ought not overwrite the memory aggressively, whereas rare, high-impact inputs should be written strongly and may even diminish the relevance of previous context.

From a generic recurrence to a linear update. A general recurrent system can be written as

$$h_t = f(h_{t-1}, x_t), \qquad y_t = g(h_t), \tag{74}$$

where f updates memory and g reads out a task-specific output. The simplest choice is to assume linear functions, yielding the (scalar) recurrence

$$h_t = ah_{t-1} + bx_t, \qquad y_t = ch_t, \tag{75}$$

where a, b, c are constants. Here, a controls *retention* (or forgetting), b controls *write strength* (how strongly the present is incorporated), and c controls the *readout* mapping from memory to output.

High-dimensional state space form (LTI SSM). In realistic settings, x_t, h_t, and y_t are vectors. For instance, x_t may include temperature, humidity, wind direction, and other features; h_t may encode aggregated statistics, trends, or other latent summaries; and y_t may be a prediction or a richer representation used downstream. We therefore take

$$x_t, y_t \in \mathbb{R}^{d_{\text{model}}}, \qquad h_t \in \mathbb{R}^{d_{\text{state}}},$$

and write the linear state space model as

$$h_t = Ah_{t-1} + Bx_t, \tag{76}$$

$$y_t = Ch_t, \tag{77}$$

where $A \in \mathbb{R}^{d_{\text{state}} \times d_{\text{state}}}$, $B \in \mathbb{R}^{d_{\text{state}} \times d_{\text{model}}}$, and $C \in \mathbb{R}^{d_{\text{model}} \times d_{\text{state}}}$ are fixed (time-invariant) parameters. Concretely,

- A governs how much of the previous memory h_{t-1} is retained;
- B governs how strongly new information x_t is written into memory;
- C governs how memory h_t is mapped to the output y_t.

A key observation: LTI implies a fixed forgetting curve and fixed write strength. The defining property of (76)–(77) is that A, B, C do *not* depend on time or on the current input. This has an immediate consequence: the model applies the *same* update rule to every day, regardless of whether the input is routine (another rainy day) or exceptional (an alien invasion).

This can be made explicit by unrolling (76):

$$h_t = A^t h_0 + \sum_{k=0}^{t} A^{t-k} B x_k. \tag{78}$$

Equation (78) shows that each past input x_k influences the current memory h_t through the factor A^{t-k}, which induces a *fixed* decay profile across time steps. In informal terms: in an LTI system, the "transparency" (importance) of older observations fades according to a pre-determined curve set by A, and the strength with which each new observation is written is set by a fixed B.

Why this is limiting: salience-sensitive memory cannot be expressed by fixed (A, B, C). The London-weather narrative highlights a mismatch between LTI memory dynamics and the demands of salient, rare events:

- For repetitive, low-salience inputs (e.g. another rainy day), it is often desirable to *write weakly* (avoid overwriting memory with redundant information) and to maintain a smooth, stable retention profile.
- For high-salience anomalies (e.g. an alien invasion), one typically wants a *strong write* (rapid incorporation into memory), and in some tasks a partial *context reset* (reducing the influence of prior "normal" context that may no longer be relevant).

However, with fixed matrices A and B, the model cannot selectively change its write/forget behaviour at $t = 5$ without also changing its behaviour on every other day. Increasing the state dimension d_{state} may increase capacity, but it does not address this core constraint: as long as the update rule remains time-invariant, the system still applies a single, global memory policy to all inputs.

From LTI to time-varying (gated) state space models. A natural resolution is to allow the state space parameters to vary with time (and, in practice, to be conditioned on the input):

$$h_t = A_t h_{t-1} + B_t x_t, \tag{79}$$

$$y_t = C_t h_t. \tag{80}$$

This time-varying formulation enables *adaptive* memory dynamics: for routine inputs, one may use smaller write strength (effectively smaller B_t) and stronger retention (effectively larger A_t); for anomalies, one may increase B_t to ensure rapid encoding and decrease A_t to reduce reliance on outdated context.

Remark (biological and modelling perspective). From a biological viewpoint, memory is gated. Novel or emotionally salient events are typically written more strongly than routine events. In particular, *traumatic* events may not only be "strongly written"; they can also reduce the influence of prior context, or even trigger a partial reset of what is stored.

From a modelling viewpoint, this suggests that the update rule should depend on the current input x_t, so that the effective parameters can change over time: $(A, B, C) \to (A_t, B_t, C_t)$. This is the core motivation for selective, time-varying mechanisms in modern sequence models such as MAMBA.

D.2 Discretisation and the Role of Δ_t

The limitation of an LTI SSM is that it applies the same memory update rule to every input. MAMBA addresses this by making the *effective* update input-dependent (hence "selective"). A convenient way to see this is to start from a continuous-time state space model and then discretise it.

Continuous-time selective SSM. We consider the continuous dynamics

$$\dot{h}(t) = Ah(t) + B(t)\, x(t), \qquad y(t) = C(t)\, h(t), \tag{81}$$

where $h(t)$ is the memory state and $x(t)$ is the input signal. Crucially, $B(t)$ and $C(t)$ are allowed to vary with time (and, in practice, with the input), while A is kept fixed.

Why A is chosen to be stable (typically negative). The homogeneous system $\dot{h}(t) = Ah(t)$ has the solution $h(t) = \exp(At)h(0)$. To prevent the state from exploding over long sequences, we require the system to be stable, i.e.

$$\mathrm{Re}(\lambda(A)) < 0 \quad \text{(in 1D this reduces to } A < 0), \tag{82}$$

so that $\exp(At)$ decays with time and past contributions do not grow without bound.

ZOH discretisation. To obtain a discrete recurrence, we discretise (81) using a zero-order hold (ZOH) assumption on each interval: over the step of length Δ_t, we treat $x(t)$ and $B(t)$ as constants equal to x_t and B_t. This yields the discrete update

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t, \tag{83}$$

with

$$\bar{A}_t = \exp(\Delta_t A), \tag{84}$$

and

$$\bar{B}_t = \left(\int_0^{\Delta_t} \exp(sA)\, ds \right) B_t = A^{-1}(\exp(\Delta_t A) - I)\, B_t, \tag{85}$$

where the last equality assumes A is invertible (the integral form is always valid).

Interpretation: Δ_t acts like a gate between "keep" and "write". Because A is stable (negative in the scalar case), increasing Δ_t makes $\Delta_t A$ more negative, hence

$$\exp(\Delta_t A) \downarrow \quad \Rightarrow \quad \bar{A}_t \text{ becomes smaller (less past is carried over).}$$

At the same time, the factor

$$\int_0^{\Delta_t} \exp(sA)\, ds$$

increases with Δ_t, so $\bar{B}_t$ becomes larger (more of the new input is written in). Therefore, Δ_t directly controls the trade-off:

- **Small Δ_t:** $\bar{A}_t \approx I$ and $\bar{B}_t$ is small $\Rightarrow$ strong retention, weak writing.
- **Large Δ_t:** $\bar{A}_t$ is small and $\bar{B}_t$ is large $\Rightarrow$ weak retention, strong writing.

London-weather example. In the London-weather story, a routine rainy day should correspond to a small Δ_t (do not overwrite memory), while a rare high-salience event (e.g. an "alien invasion") should correspond to a large Δ_t (write strongly and reduce reliance on prior context).

What remains to be designed. Up to this point, the time-varying parts are Δ_t, B_t, and C_t. In MAMBA, these quantities are produced from the current input via learnable functions:

$$\Delta_t = f(x_t), \qquad B_t = g(x_t), \qquad C_t = h(x_t). \tag{86}$$

The next subsection explains how these functions are parameterised in practice and why this yields an efficient selective update.

D.3 Input-Dependent Parametrisation in Mamba

We continue with the notation $x_t \in \mathbb{R}^{d_{\text{model}}}$ and $h_t \in \mathbb{R}^{d_{\text{state}}}$. The key step in Mamba is to produce the time-varying quantities

$$\Delta_t,\ B_t,\ C_t$$

directly from the current input x_t, i.e. $\Delta_t = f(x_t)$, $B_t = g(x_t)$, $C_t = h(x_t)$. This section describes the parameterisation used in the implementation.

Overview. Given an input token x_t, the model:
(i) expands it to a higher-dimensional internal width d_{inner},
(ii) injects local context via a causal 1D convolution,
(iii) projects the result to obtain three selective outputs: a low-rank representation of Δ_t and the vectors B_t and C_t. The overall flow is:

$$x_t \xrightarrow{\texttt{in\_proj}} (u_t, z_t) \xrightarrow{\text{causal conv + SiLU on } u_t} \tilde{u}_t \xrightarrow{\texttt{x\_proj}} (\Delta_t^{\text{raw}}, B_t, C_t) \xrightarrow{\texttt{dt\_proj + softplus}} \Delta_t.$$

D.3.1 Input expansion and the two branches

First, `in_proj` maps x_t from d_{model} to $2d_{\text{inner}}$, and we split the output into two d_{inner}-dimensional branches:

$$[u_t, z_t] = W_{\text{in}} x_t, \qquad u_t, z_t \in \mathbb{R}^{d_{\text{inner}}}. \tag{87}$$

The u_t branch carries the main content used to generate selective parameters. The z_t branch is a gate used later to modulate the output (so it can be ignored at first reading).

Remark. The expansion to d_{inner} increases the feature capacity of the *parameter generator* that will produce Δ_t, B_t, C_t.

D.3.2 Local context via causal convolution

A depthwise causal 1D convolution is applied to the u-stream, followed by a SiLU nonlinearity:

$$\tilde{u}_t = \text{SiLU}\big(\text{CausalConv1D}(u_{\le t})\big), \qquad \tilde{u}_t \in \mathbb{R}^{d_{\text{inner}}}. \tag{88}$$

This step provides a short local window of past information before any state update is computed.

Why this matters. If parameter selection depended only on the instantaneous x_t, the model would struggle to judge whether an event is "routine" or "rare" (e.g. an "alien invasion" is only anomalous relative to nearby context). The causal convolution supplies that local reference.

D.3.3 Producing (Δ_t, B_t, C_t)

Next, `x_proj` maps $\tilde{u}_t$ to a vector of dimension $d_{\text{rank}} + 2d_{\text{state}}$, which is split into three parts:

$$[\Delta_t^{\text{raw}},\, B_t,\, C_t] = W_{\text{x}}\tilde{u}_t, \tag{89}$$

where

$$\Delta_t^{\text{raw}} \in \mathbb{R}^{d_{\text{rank}}}, \qquad B_t \in \mathbb{R}^{d_{\text{state}}}, \qquad C_t \in \mathbb{R}^{d_{\text{state}}}.$$

Remark (why $d_{\text{rank}} \ll d_{\text{inner}}$). The step size Δ_t is ultimately required at width d_{inner}, but generating it directly at that width would be expensive. Mamba therefore uses a low-rank bottleneck: it predicts Δ_t^{raw} in a small dimension d_{rank}, then lifts it back to d_{inner}.

D.3.4 Δ_t projection and positivity

The low-rank Δ_t^{raw} is projected to d_{inner} via `dt_proj`:

$$s_t = W_\Delta\, \Delta_t^{\text{raw}} + b_\Delta, \qquad s_t \in \mathbb{R}^{d_{\text{inner}}}. \tag{90}$$

Finally, Mamba enforces positivity of the step size using a softplus:

$$\Delta_t = \text{softplus}(s_t), \qquad \Delta_t \in \mathbb{R}^{d_{\text{inner}}}. \tag{91}$$

Remark (why positivity). Δ_t plays the role of a discretisation step size. Ensuring $\Delta_t > 0$ avoids degenerate or unstable behaviour and makes the interpretation in the discretised update well-defined.

D.3.5 Bias initialisation and log-uniform sampling

The initial range of step sizes is chosen as $\Delta_{\min} \le \Delta_t \le \Delta_{\max}$ (e.g. 10^{-3} to 10^{-1}). Sampling Δ uniformly in *linear* space would over-represent large values. Instead, we sample Δ approximately *log-uniformly* so that each order of magnitude is represented similarly:

$$\log \Delta \sim \text{Uniform}(\log \Delta_{\min}, \log \Delta_{\max}).$$

To make $\Delta_t \approx \text{softplus}(b_\Delta)$ at initialisation (when the learned term is near zero), the bias is set via the inverse softplus:

$$b_\Delta = \text{softplus}^{-1}(\Delta_{\text{init}}) = \log\big(\exp(\Delta_{\text{init}}) - 1\big), \tag{92}$$

where Δ_{init} is sampled log-uniformly as above. This initialises the model with a diverse set of timescales: small Δ supports longer retention, and large Δ supports rapid updating.

Putting it together. At each time step, the model computes Δ_t, B_t, and C_t from x_t using the pipeline above, and then inserts them into the selective SSM update derived in Appendix D.2.

References

[1] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. https://hazyresearch.stanford.edu/blog/2022-01-14-s4-3, jan 2022. Hazy Research Blog.